

ENGINEERING REBOOT 2026

Day-15 Field Manual - Full Publication Edition

ENGINEERING REBOOT 2026

Week 03 – Data Structures & Collection Processing

Day 15 Field Manual

Personal Data Analyzer

Version 1.0 – Publication Manuscript Source

Status: REVIEW DRAFT

PHASE 0 – MISSION BRIEF

Objective

Day 15 serves as the capstone exercise for Week 03.

During Days 11 through 14, individual concepts were introduced and practiced independently:

- Lists
- Collection Traversal
- Dictionaries
- Collection Processing
- Aggregation
- Reporting

Day 15 combines all of these concepts into a single application:

personal_data_analyzer.py

The objective is to simulate how real software systems store structured information, process collect.

Core Concepts

- Lists
- Dictionaries
- Collection Traversal
- Collection Processing
- Aggregation
- Statistical Analysis
- Reporting

Engineering Applications

Employee Records

Student Records

Inventory Systems

Customer Databases

Asset Management Systems

Reporting Dashboards

Business Intelligence Systems

Learning Objectives

By the end of Day 15 you should be able to:

- Store structured information using dictionaries
- Store multiple records using lists
- Traverse collections of records
- Extract information from collections
- Calculate summary statistics
- Generate structured reports
- Combine all Week 03 concepts into a complete application

Deliverable Inventory

1. `personal_data_analyzer.py`
2. `Notes.md`
3. `Journal.md`
4. Evidence Package

PHASE 1 – PERSONAL DATA ANALYZER CORE IMPLEMENTATION

Filename

`personal_data_analyzer.py`

Objective

Create a collection of structured personal records and generate a formatted report using collection-

Implement

Create the following data structure:

```
people = [  
    ...  
    {  
        "name": "Alice",  
        "age": 25,  
        "city": "New York"  
    },  
    {  
        "name": "Bob",  
        "age": 32,  
        "city": "Chicago"  
    },  
    {  
        "name": "Charlie",  
        "age": 28,  
        "city": "Boston"  
    },  
    {  
        "name": "Diana",
```

```
        "age": 35,  
        "city": "Seattle"  
    },  
    ],
```

```
]  
  
Display a report showing:
```

- Name
- Age
- City

Suggested Processing

Traverse the collection:

for person in people:

```
    ...  
    print(person)
```

Then generate a formatted report:

Personal Data Report

Name: Alice

Age: 25

City: New York

Name: Bob

Age: 32

City: Chicago

(continue for all records)

Validation Matrix

Expected Output

Personal Data Report

Name: Alice

Age: 25

City: New York

Name: Bob

Age: 32

City: Chicago

Name: Charlie

Age: 28

City: Boston

Name: Diana

Age: 35

City: Seattle

Evidence Checkpoint

Capture:

personal_data_analyzer-Code.png

Capture:

personal_data_analyzer-T01-ExpectedResult.png

Completion Checkpoint

■ Data Structure Created

■ Collection Traversed

■ Report Generated

■ Validation Completed

■ Evidence Captured

PHASE 2 – COLLECTION PROCESSING & REPORTING

Filename

personal_data_analyzer.py

Objective

Process the collection of personal records and generate summary statistics.

This phase introduces the same collection-processing patterns practiced during Day 14, but now applied to a new dataset.

The goal is to extract meaningful information from the dataset.

Implement

Using the existing people collection:

```
people = [  
    ...  
    {"name": "Alice", "age": 25, "city": "New York"},  
    {"name": "Bob", "age": 32, "city": "Chicago"},  
    {"name": "Charlie", "age": 28, "city": "Boston"},  
    {"name": "Diana", "age": 35, "city": "Seattle"}  
]
```

Calculate:

- Total People
- Average Age
- Youngest Age
- Oldest Age

Display results in report format.

Suggested Processing

Create an age collection:

```
ages = []
```

Populate the collection:

```
for person in people:
```

```
    ...
```

```
ages.append(person["age"])
```

Calculate total records:

```
total_people = len(people)
```

Calculate average age:

```
average_age = sum(ages) / len(ages)
```

Determine youngest age:

```
youngest_age = min(ages)
```

Determine oldest age:

```
oldest_age = max(ages)
```

Validation Matrix

Expected Output

Personal Data Summary

Total People: 4

Average Age: 30.0

Youngest Age: 25

Oldest Age: 35

Evidence Checkpoint

Capture:

personal_data_analyzer-Code.png

Capture:

personal_data_analyzer-T02-ExpectedResult.png

Completion Checkpoint

■ Collection Processed

■ Statistical Analysis Completed

■ Validation Completed

■ Evidence Captured

PHASE 3 – CHALLENGE EXTENSION

Filename

personal_data_analyzer.py

Objective

Extend the Personal Data Analyzer by introducing conditional counting and category reporting.

This phase combines:

- Lists
- Dictionaries
- Traversal
- Collection Processing
- Aggregation
- Conditional Logic

Implement

Determine:

- People Age 30 And Above
- People Below Age 30

Display a category report.

Suggested Processing

Initialize counters:

```
age_30_and_above = 0
```

```
below_30 = 0
```

Traverse records:

```
for person in people:
```

```
    ...
```

```
    if person["age"] >= 30:
```

```
        age_30_and_above += 1
```

```
    else:
```

```
        below_30 += 1
```

```
    ...
```

Display results.

Validation Matrix

Expected Output

Age Distribution Report

```
---
```

Age 30 And Above: 2

Below Age 30: 2

Challenge Extension 2

Determine the number of people residing in each city.

Suggested Logic

```
city_count = {}
```

```
for person in people:
```

```
    ...
```

```
    city = person["city"]
```

```
if city in city_count:
```

```
    city_count[city] += 1
```

```
else:
```

```
    city_count[city] = 1
```

```
...
```

Display:

City Distribution

```
---
```

New York : 1

Chicago : 1

Boston : 1

Seattle : 1

Validation Matrix – Challenge Extension

Expected Result

City Distribution

New York : 1

Chicago : 1

Boston : 1

Seattle : 1

Evidence Checkpoint

Capture:

personal_data_analyzer-Code.png

Capture:

personal_data_analyzer-T03-ExpectedResult.png

Completion Checkpoint

■ Conditional Counting Completed

■ City Analysis Completed

■ Challenge Extension Completed

■ Validation Completed

■ Evidence Captured

Phase Completion Summary

Upon completion of Phase 3, the application will have demonstrated:

■ List Processing

■ Dictionary Processing

■ Collection Traversal

■ Aggregation

■ Statistical Analysis

■ Conditional Counting

■ Structured Reporting

These collectively satisfy the technical objectives established for Week 03 and prepare for transition

Files, Modules & Persistence.

PHASE 4 – NOTES

Create

03-Week-03/Day-15/Notes/Notes.md

Objective

Document concepts learned during Day 15.

Required Sections

- Concepts Learned
- Key Commands
- Key Observations
- Lessons Learned
- Questions For Future Study

Suggested Topics

What is a dictionary?

How do lists and dictionaries work together?

How are structured records represented?

How can collections of records be traversed?

How can statistical information be extracted from collections?

How can aggregation techniques be applied to structured data?

How are category counts calculated?

Where do data-analysis techniques appear in real-world software systems?

Notes Template

Concepts Learned

Key Commands

Key Observations

Lessons Learned

Questions For Future Study

Completion Checkpoint

■ Notes Completed

PHASE 5 – JOURNAL

Create

03-Week-03/Day-15/Journal/Journal.md

Objective

Capture personal reflections regarding Day 15 execution.

Required Sections

- Summary
- Challenges Encountered
- Solutions Applied
- Confidence Assessment
- Reflection
- Tomorrow Preparation

Suggested Reflection Questions

How did processing structured records differ from processing simple collections?

Which portion of the Personal Data Analyzer felt most intuitive?

Which concept required the most effort to understand?

How comfortable are you using dictionaries?

How comfortable are you combining multiple data structures in a single application?

Where might you encounter similar data-analysis patterns in professional software systems?

How prepared do you feel for Week 04?

Journal Template

Summary

Challenges Encountered

Solutions Applied

Confidence Assessment

Reflection

Tomorrow Preparation

Completion Checkpoint

■ Journal Completed

PHASE 6 – GIT OPERATIONS

Objective

Preserve Day 15 artifacts in source control and verify repository synchronization.

Execute

git status

git add .

git commit -m "Complete Day 15 personal data analyzer"

git push

git fetch

git status

git branch -vv

Preferred End State

Your branch is up to date with 'origin/main'

nothing to commit, working tree clean

Evidence Checkpoint

Capture:

Git-Operations.png

Must Show

git add .

git commit

git push

git fetch

git status

git branch -vv

Completion Checkpoint

■ Git Operations Completed

■ Repository Synchronized

■ Evidence Captured

END-OF-DAY SUCCESS CRITERIA

■ personal_data_analyzer.py completed

■ Personal Data Report completed

■ Personal Data Summary completed

■ Age Distribution Report completed

■ City Distribution Report completed

■ All validation completed

■ Exercise evidence captured

■ Notes completed

■ Journal completed

■ Git workflow completed

■ Git-Operations.png captured

■ Remote synchronization verified

■ Working tree clean

MISSION COMPLETE ✓

Congratulations.

You have successfully completed Day 15:

Personal Data Analyzer

You have now demonstrated the ability to:

- Create structured records using dictionaries
- Store multiple records in collections
- Traverse collections of structured data
- Perform aggregation operations
- Calculate statistical summaries
- Perform conditional counting
- Generate structured reports
- Combine all Week 03 concepts into a single application

WEEK 03 COMPLETE ✓

You have successfully completed:

- Day 11 – Lists and Collections
- Day 12 – Collection Traversal & Iteration
- Day 13 – Dictionaries & Structured Data
- Day 14 – Collection Processing & Analysis
- Day 15 – Personal Data Analyzer

Week 03 Learning Outcomes Achieved

■ Lists

■ Collection Traversal

■ Dictionaries

- Collection Processing
- Aggregation
- Reporting
- Structured Data Analysis

WEEK 04 PREVIEW

Topic

Files, Modules & Persistence

Expected Deliverables

file_reader.py

file_writer.py

csv_processor.py

Core Concepts

- File Reading
- File Writing
- Text Files
- CSV Files
- Modules
- Code Reuse
- Persistence

Engineering Focus

Learn how software systems persist information beyond program execution and begin organizing code in